Distributed File System (DFS)

1. Overview of the DFS Simulation

This Python program simulates a Distributed File System (DFS) similar to Google File System (GFS) and Hadoop Distributed File System (HDFS). It demonstrates core DFS concepts:

- **DataNode:** Represents a storage node that holds chunks of data.
- **NameNode:** Central metadata server that tracks which chunks are on which nodes.
- **DistributedFileSystem:** Orchestrates file writes, reads, deletions, replication, and fault recovery.

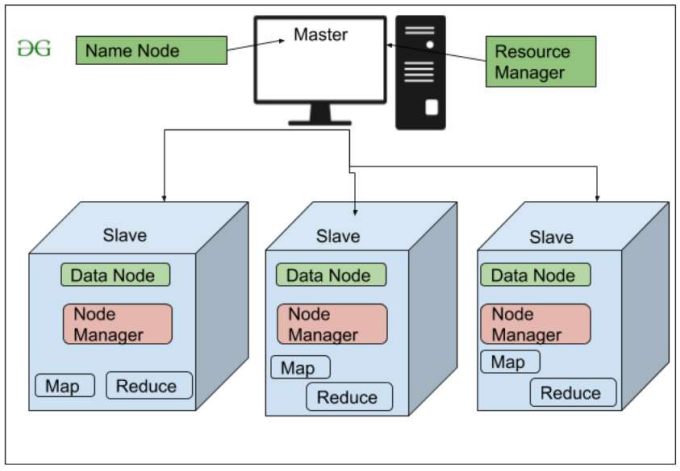

Fig : HDFS

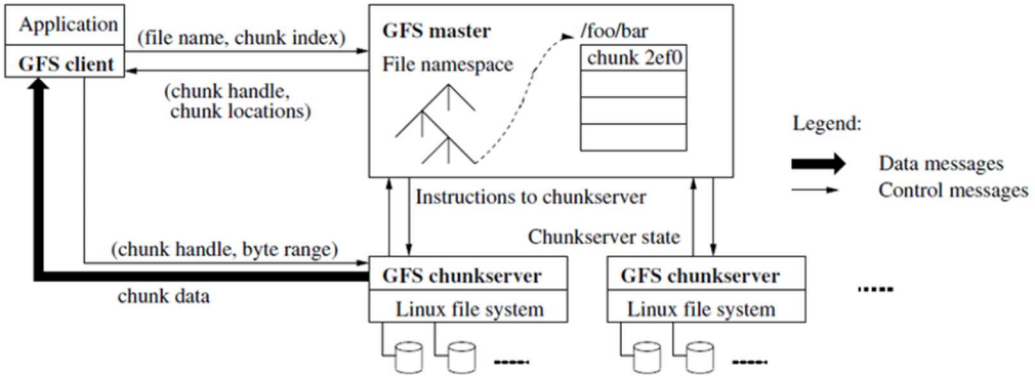

Fig : GFS

2. DataNode Class

The DataNode class models a single storage server in the cluster. Each node has an ID, a list of stored chunk IDs, and an active/inactive status.

2.1 Constructor — __init__

→ *Initialize a DataNode with a unique ID, an empty chunk list, and mark it as active.*

```
class DataNode:
    def __init__(self, node_id):
        self.node_id = node_id   # Unique identifier for this node
        self.chunks = []          # List of chunk IDs stored on this node
        self.active = True        # Node starts in active (healthy) state
```

2.2 store_chunk(chunk_id)

→ *Store a chunk on this node — raise an error if the node is down, skip if already stored.*

```
    def store_chunk(self, chunk_id):
        if not self.active:
            raise ValueError(f'Node {self.node_id} is inactive.')
        if chunk_id not in self.chunks:
            self.chunks.append(chunk_id)
```

2.3 get_chunk(chunk_id)

→ *Retrieve a chunk from this node — raises errors if node is down or chunk is missing.*

```
    def get_chunk(self, chunk_id):
        if not self.active:
            raise ValueError(f'Node {self.node_id} is inactive.')
        if chunk_id in self.chunks:
            return chunk_id
        raise ValueError(f'Chunk {chunk_id} not found on Node {self.node_id}.')
```

2.4 delete_chunk(chunk_id)

→ *Remove a specific chunk from the node's storage list, raising errors as needed.*

```
    def delete_chunk(self, chunk_id):
        if not self.active:
            raise ValueError(f'Node {self.node_id} is inactive.')
        if chunk_id in self.chunks:
            self.chunks.remove(chunk_id)
        else:
```

```
            raise ValueError(f'Chunk {chunk_id} not found on Node {self.node_id}.')
```

2.5 fail()

→ Simulate a node crash — mark it inactive and wipe all its stored chunks.

```
    def fail(self):
        self.active = False   # Node goes offline
        self.chunks = []      # All data is lost (simulating hardware failure)
```

3. NameNode Class

The NameNode is the master metadata server. It never stores actual file data — it only stores a mapping of filename → list of (chunk_id, [node_ids]) pairs. This is analogous to the NameNode in HDFS.

3.1 Constructor — __init__

→ *Initialize an empty metadata dictionary to track all files and their chunk locations.*

```
class NameNode:
    def __init__(self):
        self.metadata = {}  # { filename: [(chunk_id, [node_ids]), ...] }
```

3.2 add_file(filename, chunks, node_assignments)

→ *Register a new file in the metadata store, pairing each chunk with its assigned nodes.*

```
    def add_file(self, filename, chunks, node_assignments):
        if filename in self.metadata:
            raise ValueError(f'File {filename} already exists.')
        self.metadata[filename] = list(zip(chunks, node_assignments))
```

3.3 get_chunk_locations(filename)

→ *Look up and return the full list of (chunk_id, [node_ids]) pairs for a given file.*

```
    def get_chunk_locations(self, filename):
        if filename not in self.metadata:
            raise ValueError(f'File {filename} not found.')
        return self.metadata[filename]
```

3.4 update_chunk_locations(filename, chunk_id, new_node_ids)

→ *After redistribution, update the metadata to reflect the new set of nodes holding a chunk.*

```
    def update_chunk_locations(self, filename, chunk_id, new_node_ids):
        if filename not in self.metadata:
            raise ValueError(f'File {filename} not found.')
        for i, (cid, nodes) in enumerate(self.metadata[filename]):
            if cid == chunk_id:
                self.metadata[filename][i] = (cid, new_node_ids)
                break
```

3.5 delete_file(filename)

→ *Remove a file's entire metadata entry from the NameNode when the file is deleted.*

```
    def delete_file(self, filename):
        if filename not in self.metadata:
            raise ValueError(f'File {filename} not found.')
        del self.metadata[filename]
```

4. DistributedFileSystem Class

This is the main orchestrator. It ties DataNodes and the NameNode together, and exposes the top-level file system operations: write, read, delete, node management, and fault recovery.

4.1 Constructor — __init__

→ Set up the DFS with a chunk size, replication factor, empty node registry, and a fresh NameNode.

```
class DistributedFileSystem:
    def __init__(self, chunk_size, replication_factor):
        if chunk_size <= 0 or replication_factor <= 0:
            raise ValueError('Chunk size and replication factor must be positive.')
        self.chunk_size = chunk_size          # Max size (KB) of each chunk
        self.replication_factor = replication_factor  # Copies per chunk
        self.data_nodes = {}                  # { node_id: DataNode }
        self.name_node = NameNode()           # Single metadata server
```

4.2 add_node(node_id)

→ Register a new DataNode in the cluster, then rebalance chunks across all nodes.

```
    def add_node(self, node_id):
        if node_id in self.data_nodes:
            raise ValueError(f'Node {node_id} already exists.')
        self.data_nodes[node_id] = DataNode(node_id)
        print(f'Node {node_id} added successfully.')
        self.redistribute_chunks()  # Rebalance chunks to include new node
```

4.3 remove_node(node_id)

→ Gracefully remove a node — simulate its failure and trigger redistribution to restore replication.

```
    def remove_node(self, node_id):
        if node_id not in self.data_nodes:
            raise ValueError(f'Node {node_id} does not exist.')
        self.data_nodes[node_id].fail()    # Mark node as down
        del self.data_nodes[node_id]       # Remove from registry
        print(f'Node {node_id} removed successfully.')
        self.redistribute_chunks()          # Recover lost replicas
```

4.4 write_file(filename, file_size)

→ Split a file into chunks, randomly assign each chunk to 'replication_factor' nodes, and register the mapping in the NameNode.

```
    def write_file(self, filename, file_size):
        try:
            # Step 1: Calculate number of chunks needed
```

```
            num_chunks = (file_size + self.chunk_size - 1) // self.chunk_size
            chunks = [f'{filename}_C{i+1}' for i in range(num_chunks)]

            # Step 2: For each chunk, pick random active nodes to store replicas
            node_assignments = []
            for i in range(num_chunks):
                available_nodes = [nid for nid, node in self.data_nodes.items() if
node.active]
                if len(available_nodes) < self.replication_factor:
                    raise ValueError('Not enough active nodes for replication.')
                assigned_nodes = random.sample(available_nodes,
self.replication_factor)
                node_assignments.append(assigned_nodes)
                for node_id in assigned_nodes:
                    self.data_nodes[node_id].store_chunk(chunks[i])

            # Step 3: Record file metadata in NameNode
            self.name_node.add_file(filename, chunks, node_assignments)
            print(f'File {filename} written. Chunks: {chunks}')
        except Exception as e:
            print(f'Error writing file {filename}: {e}')
```

4.5 read_file(filename)

→ Look up chunk locations from the NameNode, then fetch each chunk from any available replica node.

```
    def read_file(self, filename):
        try:
            # Step 1: Get chunk → node mappings from NameNode
            chunk_locations = self.name_node.get_chunk_locations(filename)
            retrieved_chunks = []

            for chunk_id, node_ids in chunk_locations:
                chunk_found = False
                random.shuffle(node_ids)   # Load balance across replicas
                for node_id in node_ids:
                    try:
                        chunk = self.data_nodes[node_id].get_chunk(chunk_id)
                        retrieved_chunks.append(chunk)
                        chunk_found = True
                        break   # Got it from one replica — stop trying
                    except ValueError:
                        continue   # Try the next replica

                if not chunk_found:
                    # All replicas failed — attempt recovery
                    self.redistribute_chunks()
                    raise ValueError(f'Chunk {chunk_id} not available on any
node.')

            print(f'File {filename} read. Chunks: {retrieved_chunks}')
            return retrieved_chunks
        except Exception as e:
            print(f'Error reading file {filename}: {e}')
            return []
```

4.6 delete_file(filename)

→ Remove all chunk copies from every DataNode that holds them, then wipe the file's metadata from the NameNode.

```
    def delete_file(self, filename):
        try:
            chunk_locations = self.name_node.get_chunk_locations(filename)
            for chunk_id, node_ids in chunk_locations:
                for node_id in node_ids:
                    try:
                        self.data_nodes[node_id].delete_chunk(chunk_id)
                    except ValueError as e:
                        print(f'Warning: Could not delete chunk {chunk_id}: {e}')
            # Finally, remove the file's entry from the NameNode
            self.name_node.delete_file(filename)
            print(f'File {filename} deleted successfully.')
        except Exception as e:
            print(f'Error deleting file {filename}: {e}')
```

4.7 simulate_node_failure(node_id)

→ Crash a node on demand — mark it failed and immediately trigger redistribution to restore lost replicas.

```
    def simulate_node_failure(self, node_id):
        try:
            if node_id not in self.data_nodes:
                raise ValueError(f'Node {node_id} does not exist.')
            self.data_nodes[node_id].fail()   # Wipes node's chunks, marks inactive
            print(f'Node {node_id} has failed.')
            self.redistribute_chunks()         # Recover replicas on surviving
nodes
        except Exception as e:
            print(f'Error simulating node failure: {e}')
```

4.8 redistribute_chunks()

→ Scan every file's chunks, find under-replicated ones, copy them from a healthy source node to new nodes, and update the NameNode metadata.

```
    def redistribute_chunks(self):
        try:
            available_nodes = [nid for nid, node in self.data_nodes.items() if
node.active]
            if len(available_nodes) < self.replication_factor:
                print('Warning: Not enough active nodes to maintain replication
factor.')
                return

            for filename in self.name_node.metadata.copy():
                for i, (chunk_id, node_ids) in
enumerate(self.name_node.get_chunk_locations(filename)):
                    # Find nodes that still have this chunk and are alive
                    active_node_ids = [nid for nid in node_ids
```

```
                                    if nid in self.data_nodes and
self.data_nodes[nid].active]

                    if len(active_node_ids) < self.replication_factor:
                        # Find a source node that actually has the chunk
                        source_node = None
                        for nid in active_node_ids:
                            try:
                                self.data_nodes[nid].get_chunk(chunk_id)
                                source_node = nid
                                break
                            except ValueError:
                                continue

                        if not source_node:
                            print(f'Warning: Chunk {chunk_id} of {filename} is
unavailable.')
                            continue

                        # Pick new nodes to fill the replication gap
                        needed = self.replication_factor - len(active_node_ids)
                        new_nodes = [nid for nid in available_nodes
                                     if nid not in active_node_ids][:needed]
                        for nid in new_nodes:
                            self.data_nodes[nid].store_chunk(chunk_id)

                        # Update NameNode with the new replica locations
                        new_node_ids = active_node_ids + new_nodes
                        self.name_node.update_chunk_locations(filename, chunk_id,
new_node_ids)
                        print(f'Redistributed chunk {chunk_id} -> {new_node_ids}')
        except Exception as e:
            print(f'Error redistributing chunks: {e}')
```

5. main() — Interactive Driver

The main() function ties everything together with user prompts. It creates a DFS, adds nodes, writes a file, reads it back, simulates a node failure, adds a new node, reads again, and finally deletes the file.

5.1 Initialize DFS

→ *Prompt the user for chunk size and replication factor, then construct the DFS object.*

```
chunk_size = int(input('Enter chunk size (in KB): '))
replication_factor = int(input('Enter replication factor: '))
dfs = DistributedFileSystem(chunk_size, replication_factor)
```

5.2 Add Initial Nodes

→ *Ask how many nodes to start with, then add them one by one (Node1, Node2, ..., NodeN).*

```
num_nodes = int(input('Enter initial number of data nodes: '))
for i in range(1, num_nodes + 1):
    dfs.add_node(f'Node{i}')
```

5.3 Write a File

→ *Take a filename and size, split the file into chunks, and distribute replicas across nodes.*

```
filename = input('Enter filename to write: ')
file_size = int(input('Enter file size (in KB): '))
dfs.write_file(filename, file_size)
```

5.4 Read the File

→ *Read back the file to confirm all chunks can be retrieved from the cluster.*

```
dfs.read_file(filename)
```

5.5 Simulate Node Failure and Re-read

→ *Crash a chosen node, trigger automatic re-replication, then confirm the file is still readable.*

```
node_id = input('Enter node to fail (e.g., Node1): ')
dfs.simulate_node_failure(node_id)
dfs.read_file(filename)   # Should still succeed — replicas on other nodes
```

5.6 Add New Node and Re-read

→ Bring a new node online, rebalance chunks to include it, then verify the file again.

```
new_node_id = input('Enter new node ID to add (e.g., Node4): ')
dfs.add_node(new_node_id)
dfs.read_file(filename)
```

5.7 Delete the File

→ Remove all chunks from DataNodes and purge the file's metadata from the NameNode.

```
dfs.delete_file(filename)
```

6. Sample Run

Below is a sample terminal session demonstrating the simulation with chunk_size=50, replication_factor=2, 3 nodes, and a 120 KB file.

```
Enter chunk size (in KB): 50
Enter replication factor: 2
Enter initial number of data nodes: 3
Node Node1 added successfully.
Node Node2 added successfully.
Node Node3 added successfully.
Enter filename to write: report
Enter file size (in KB): 120
File report written successfully. Chunks: ['report_C1','report_C2','report_C3']
Assignments: [['Node1','Node3'], ['Node2','Node3'], ['Node1','Node2']]
File report read successfully. Retrieved chunks:
['report_C1','report_C2','report_C3']
Enter node to fail (e.g., Node1): Node1
Node Node1 has failed.
Redistributed chunk report_C1 of report to ['Node3', 'Node2']
Redistributed chunk report_C3 of report to ['Node3', 'Node2']
File report read successfully. Retrieved chunks:
['report_C1','report_C2','report_C3']
Enter new node ID to add (e.g., Node4): Node4
Node Node4 added successfully.
File report read successfully. Retrieved chunks:
['report_C1','report_C2','report_C3']
File report deleted successfully.
```